PRODUCT REQUIREMENTS DOCUMENT

GenAI Studio

Evaluate, Track & Optimize AI-Generated Code Adoption

Version 1.0 | March 2026 | Engineering Productivity

Document Overview

Product Name	AI Code Intelligence Dashboard
Version	1.0
Status	Draft for Review
Date	March 2026
Prepared For	Claude Code Agentic Build
Audience	Engineering Leaders, DevOps, Platform Teams
Target Tools	Cursor, GitHub Copilot, and future AI coding assistants

1. EXECUTIVE SUMMARY

Engineering organizations are rapidly adopting AI-assisted coding tools such as Cursor and GitHub Copilot. While these tools accelerate development velocity, engineering leaders lack visibility into a critical set of questions: How much of the code being written is AI-generated? How much AI-generated code actually makes it to production? Which developers and teams are adopting AI tooling effectively?

The AI Code Intelligence Dashboard is a real-time analytics platform designed to answer these questions at every level of the organization — from individual contributors to entire engineering departments. By integrating directly with version control systems and AI coding tool telemetry, the dashboard provides transparent, actionable metrics on the split between manual and AI-generated code, merge rates, and per-user adoption patterns.

2. PROBLEM STATEMENT

2.1 The Challenge

Despite widespread adoption of AI coding assistants, engineering teams face a fundamental observability gap:

- No unified view of AI vs. manual code contribution across repositories
- Inability to measure how much AI-generated code is being accepted and merged into live environments
- No per-user attribution of AI tool usage or merge behavior
- Difficulty benchmarking the ROI of AI tool licenses (Copilot, Cursor, etc.)
- Risk of unreviewed or low-quality AI-generated code reaching production undetected

2.2 Impact

Without this visibility, organizations cannot make data-driven decisions about AI tool investment, developer training, or code review processes. Engineering managers lack the evidence to coach developers on effective AI usage, and finance teams cannot validate the return on significant AI tooling expenditure.

3. GOALS & OBJECTIVES

3.1 Primary Goals

- Provide real-time visibility into the ratio of AI-generated vs. manually written code across all live repositories
- Track and display the percentage of AI-generated code that is merged into production environments
- Deliver per-user analytics to understand individual AI tool adoption and merge behavior
- Support multiple AI coding tools (Cursor, GitHub Copilot) with a unified reporting layer
- Enable engineering leaders to set targets, track trends, and generate reports on AI code adoption

3.2 Success Metrics

Metric	Definition	Target at Launch
AI Code Detection Accuracy	% of AI-generated code correctly identified	>= 90%
Dashboard Latency	Time from commit to dashboard update	< 5 minutes

User Coverage	% of engineering users with active tracking	100% of opted-in users
Merge Rate Accuracy	Accuracy of AI-to-merge attribution	>= 95%
Tool Integration Coverage	AI tools with native integration at launch	Cursor, GitHub Copilot

4. SCOPE & BOUNDARIES

4.1 In Scope

- Real-time code scanning and AI attribution engine
- Integration with GitHub, GitLab, and Bitbucket (pull request and commit data)
- Native integrations with Cursor and GitHub Copilot telemetry APIs
- Organization-level, team-level, and user-level dashboard views
- Merge rate tracking: AI code accepted into PRs and merged to main/production branches
- Historical trend analysis (rolling 7, 30, 90-day windows)
- Exportable reports (CSV, PDF)
- Role-based access control (Admin, Manager, Developer)

4.2 Out of Scope (v1.0)

- Code quality scoring or static analysis of AI-generated code
- Integration with AI tools beyond Cursor and GitHub Copilot (planned for v2.0)
- Real-time IDE plugin or in-editor overlays
- Automated code review or AI code rejection workflows

5. USER PERSONAS

5.1 Engineering Leader / VP of Engineering

Needs an org-wide view of AI adoption to report ROI to leadership. Wants to understand which teams are leveraging AI effectively and which are underutilizing their tool licenses. Requires trend data over time and export capability for board-level reporting.

5.2 Engineering Manager / Team Lead

Wants team-level breakdowns to coach developers. Needs to see per-user AI vs. manual code ratios and merge success rates. Uses the dashboard to identify developers who may need support in working effectively with AI-generated code and to recognize high performers.

5.3 Developer / Individual Contributor

Wants to understand their own AI usage patterns and how their AI-generated code performs through the review and merge pipeline. May use personal stats to improve their workflow and demonstrate productivity improvements during performance reviews.

5.4 DevOps / Platform Engineer

Responsible for deploying and maintaining the dashboard infrastructure. Needs clear integration documentation, webhook and API access, and a reliable data pipeline. Monitors system health and data freshness of the dashboard itself.

6. FEATURES & REQUIREMENTS

6.1 AI Code Detection Engine

6.1.1 Functionality

- Analyze every commit and pull request across connected repositories
- Classify each code block or file diff as AI-generated or manually written
- Use a combination of: AI tool telemetry data (Copilot, Cursor events), heuristic pattern recognition (code structure, comment styles), and direct API attribution where available
- Assign a confidence score (High / Medium / Low) to each classification

6.1.2 AI Tool Integrations

- GitHub Copilot: Ingest Copilot telemetry via GitHub Copilot Business/Enterprise API for accepted suggestions per user and file
- Cursor: Ingest Cursor AI completion events via Cursor telemetry or workspace logs
- Both integrations must attribute generated code to specific users, repositories, and branches

6.2 Live Code Percentage View

- Display real-time and rolling-average breakdowns of AI vs. manual code by: Organization, Team / Department, Repository, User
- Metric: % AI-Generated Lines of Code (LoC) vs. % Manual LoC in production branch
- Visual representation: stacked bar charts, donut charts, and trend line graphs

- Filter by: date range, repository, team, AI tool source (Copilot vs. Cursor)

6.3 Merge Rate Analytics

- Track the full lifecycle of AI-generated code: Written (detected in feature branch) > Submitted (included in PR) > Merged (accepted into main/production)
- Calculate and display: AI Code Merge Rate = (AI LoC Merged) / (AI LoC Written) x 100%
- Compare AI merge rate vs. manual code merge rate to understand review dynamics
- Surface PRs with high concentrations of AI-generated code that were rejected or abandoned

6.4 User-Level Dashboard

- Per-developer scorecard showing: Total commits in period, % AI-generated code, % AI code merged, Primary AI tool used, Trend vs. prior period
- Leaderboard view (opt-in) to highlight AI power users
- Developer self-service view: each user can see their own stats without requiring manager access
- Manager view: access to full team roster with all individual metrics

6.5 Organization & Team Dashboards

- Hierarchical drill-down: Organization > Division > Team > User
- Configurable KPI tiles for at-a-glance metrics: Org AI adoption %, Top AI-adopting team, Highest merge rate user, AI tool cost efficiency (LoC per license)
- Heatmap of AI adoption across teams over time
- Alerts / threshold notifications when AI adoption drops below or exceeds configured benchmarks

6.6 Reporting & Export

- Scheduled weekly/monthly PDF email reports for managers and leaders
- On-demand CSV export of all underlying metrics
- API endpoint for downstream BI tool integration (e.g., Tableau, Looker, PowerBI)

7. DASHBOARD UX & DESIGN PRINCIPLES

The dashboard must adhere to the following design principles to ensure adoption and usability across all technical and non-technical users:

Principle	Description
Clarity First	Every metric must include a clear label, unit, and time period. No ambiguous numbers.
Progressive Disclosure	High-level summaries load first; users drill down for detail on demand.
Real-Time Feel	Data refreshes every 5 minutes. Timestamps always visible to indicate freshness.
Role-Aware Views	UI surfaces only what is relevant to the logged-in user's role.
Mobile-Responsive	Dashboard must be fully functional on tablet and desktop. Mobile is read-only.
Accessible	WCAG 2.1 AA compliance. All charts must have tabular data alternatives.

7.1 Key Dashboard Screens

- Home / Overview: Org-wide KPI tiles, AI vs. manual trend chart, top teams and users
- Code Breakdown: Repository and branch-level AI % view with filter controls
- Merge Analytics: Funnel view from written > submitted > merged, with AI vs. manual comparison
- User Profiles: Per-developer scorecard with full metric history
- Team View: Manager's team roster table with sortable columns and export
- Settings: Repository connections, AI tool integrations, user role management, thresholds

8. TECHNICAL ARCHITECTURE

8.1 Architecture Overview

The platform is composed of four primary layers:

Layer	Component	Responsibility
Ingestion	VCS Webhooks (GitHub, GitLab, Bitbucket)	Stream commit and PR events in real time
Ingestion	AI Tool Telemetry Connectors	Pull Copilot and Cursor completion events
Processing	Attribution Engine	Classify code as AI or manual, attach user and tool metadata

Processing	Metrics Aggregator	Compute LoC counts, percentages, merge rates by dimension
Storage	Time-Series Data Store	Store metric snapshots for trend analysis
Presentation	REST API + GraphQL	Serve dashboard data to frontend
Presentation	React Dashboard Frontend	Render visualizations and interactive views

8.2 Data Flow

- Developer commits code or accepts AI suggestion in IDE
- VCS webhook fires on push/PR event; AI tool telemetry event logged
- Attribution Engine ingests both signals, correlates by user, repo, file, and timestamp
- Metrics Aggregator computes and stores updated statistics
- Dashboard Frontend polls API every 5 minutes and updates live views

8.3 Integration Requirements

- GitHub: GitHub Apps (OAuth) with read access to repos, pull requests, and Copilot Business telemetry
- GitLab: GitLab OAuth + webhooks for push and merge request events
- Bitbucket: Bitbucket OAuth + repository webhooks
- Cursor: Cursor workspace telemetry API (where available) or local log ingestion agent
- GitHub Copilot: GitHub Copilot for Business seat-level telemetry API

9. SECURITY & PRIVACY

- All VCS tokens and AI tool credentials stored encrypted at rest (AES-256)
- Data in transit encrypted via TLS 1.2+
- Role-based access control: Developers see only their own data; Managers see their team; Admins see all
- No source code content is stored — only metadata (file paths, LoC counts, diff sizes, timestamps, attribution flags)
- Compliance with SOC 2 Type II controls
- GDPR-compliant data handling: user data deletable on request
- Audit log of all admin actions and data exports
- Optional: anonymized/aggregated mode for organizations where individual-level tracking raises HR concerns

10. PHASED DELIVERY PLAN

Phase	Milestone	Timeline	Key Deliverables
Phase 1	Foundation	Month 1-2	VCS integrations (GitHub), Attribution Engine v1, Basic org + user dashboard
Phase 2	AI Tool Integration	Month 2-3	GitHub Copilot telemetry, Cursor integration, Live AI % view, Merge rate analytics
Phase 3	Advanced Analytics	Month 3-4	Team dashboards, trend analysis, leaderboard, threshold alerts, CSV/PDF export
Phase 4	Scale & API	Month 4-5	GitLab + Bitbucket, REST/GraphQL API, BI tool connector, GitLab merge analytics
Phase 5	Polish & Launch	Month 5-6	Mobile responsiveness, WCAG compliance, onboarding flow, documentation, GA launch

11. RISKS & MITIGATIONS

Risk	Likelihood	Impact	Mitigation
AI tool APIs limit telemetry access	Medium	High	Build fallback heuristic detection; engage Cursor/Copilot for partner API access
Developer privacy concerns re: individual tracking	High	Medium	Offer anonymized-aggregate mode; clear opt-in communication; HR alignment before rollout
VCS webhook reliability at scale	Medium	Medium	Implement event queue with retry logic; dead-letter queue for failed events

Attribution accuracy below 90%	Low	High	Confidence scoring and human review mode for flagged edge cases
Scope creep into code quality features	High	Medium	Strict v1.0 scope boundary enforced in sprint planning; quality features roadmapped to v2.0

12. APPENDIX — KEY METRICS GLOSSARY

Term	Definition
AI LoC	Lines of code classified as AI-generated in a given period and scope
Manual LoC	Lines of code written without AI assistance
AI Code %	AI LoC / (AI LoC + Manual LoC) x 100
AI Merge Rate	AI LoC merged to production / AI LoC written x 100
Adoption Rate	% of developers actively using at least one AI tool in a given period
Suggestion Accept Rate	AI suggestions accepted / AI suggestions shown (sourced from tool telemetry)
Live Environment	Production or main branch of a repository (configurable per repo)
Attribution Confidence	High / Medium / Low score assigned by the Attribution Engine to each classification
Fast Tokens (Cursor)	Tokens used for inline autocomplete and quick completions in Cursor
Slow Tokens (Cursor)	Tokens used for longer agentic tasks, refactors, and multi-file edits in Cursor
License Idle	A paid AI tool seat assigned but not activated within 30 days
PR Merge Rate	Percentage of opened pull requests successfully merged (all code, not AI-specific)

13. SYNTHETIC DATA — SAMPLE DASHBOARD STATE (March 2026)

The following synthetic dataset illustrates a realistic snapshot of the dashboard for a mid-sized engineering organization (TechCorp Inc.) with 3 teams and 6 active developers. All data covers the trailing 30-day period ending March 20, 2026.

13.1 Organization Summary

Org Metric	Value	vs. Prior 30 Days
Total Developers (Licensed)	6	—
Active AI Tool Users	5 of 6 (83%)	+1 user (+17%)
Total Lines of Code Committed	124,800 LoC	+8,200 LoC
AI-Generated LoC	61,152 LoC	+14,300 LoC
Manual LoC	63,648 LoC	-6,100 LoC
Org-Wide AI Code %	49.0%	+10.2 pp
AI Code Merge Rate (to main)	73.4%	+5.8 pp
Total AI Tokens Used (all tools)	4,218,500 tokens	+820,000 tokens
GitHub Copilot Suggestions Shown	38,400	+4,100
GitHub Copilot Accept Rate	61.2%	+3.4 pp
Cursor AI Completions Accepted	22,750	+5,200

13.2 Team Breakdown

TechCorp Inc. has three engineering teams contributing to the dashboard data period:

Team	Head Count	AI Users	AI Code %	AI Merge Rate	Primary Tool	Total Tokens Used
Platform Engineering	2	2 / 2 (100%)	62.1%	81.0%	Cursor	1,840,000
Frontend Product	2	2 / 2 (100%)	51.4%	74.2%	GitHub Copilot	1,510,000
Data & ML Engineering	2	1 / 2 (50%)	28.3%	58.5%	GitHub Copilot	868,500

Insight: Platform Engineering leads on both AI adoption and merge success, driven by Cursor-heavy workflows. Data & ML Engineering shows the lowest adoption — one developer (Priya Nair) has not yet activated an AI tool license.

13.3 User-Level Profiles

Below are individual developer scorecards for the 30-day period. Each profile includes token consumption, AI vs. manual code split, merge outcomes, and primary tool usage.

User 1 | Jordan Lee — Senior Engineer, Platform Engineering

Metric	Value
Team	Platform Engineering
Primary AI Tool	Cursor
Total Commits (30d)	84 commits
Total LoC Written	26,400 LoC
AI-Generated LoC	18,480 LoC (70.0% of total)
Manual LoC	7,920 LoC (30.0% of total)
AI LoC Merged to Main	15,893 LoC (AI Merge Rate: 86.0%)
Cursor Tokens Used	1,120,000 tokens (Fast: 780,000 | Slow: 340,000)
Cursor Completions Accepted	14,200 (Accept Rate: 74.3%)
PRs Opened / Merged	21 opened / 19 merged (PR Merge Rate: 90.5%)
Org Leaderboard Rank	#1 — Power User | Token usage trending +18% MoM

User 2 | Alex Reyes — Staff Engineer, Platform Engineering

Metric	Value
Team	Platform Engineering
Primary AI Tool	Cursor + GitHub Copilot (dual-tool user)
Total Commits (30d)	61 commits
Total LoC Written	19,800 LoC
AI-Generated LoC	10,692 LoC (54.0% of total)
Manual LoC	9,108 LoC (46.0% of total)

AI LoC Merged to Main	8,982 LoC (AI Merge Rate: 84.0%)
Cursor Tokens Used	720,000 tokens (Fast: 540,000 | Slow: 180,000)
Copilot Tokens Used	310,000 tokens
Copilot Suggestions Shown / Accepted	8,200 shown / 5,330 accepted (Accept Rate: 65.0%)
Cursor Completions Accepted	5,600 (Accept Rate: 68.0%)
PRs Opened / Merged	15 opened / 13 merged (PR Merge Rate: 86.7%)
Org Leaderboard Rank	#2 — Power User | Copilot for boilerplate, Cursor for architecture

User 3 | Sam Okafor — Frontend Engineer II, Frontend Product

Metric	Value
Team	Frontend Product
Primary AI Tool	GitHub Copilot
Total Commits (30d)	73 commits
Total LoC Written	22,600 LoC
AI-Generated LoC	12,656 LoC (56.0% of total)
Manual LoC	9,944 LoC (44.0% of total)
AI LoC Merged to Main	9,617 LoC (AI Merge Rate: 76.0%)
Copilot Tokens Used	580,000 tokens
Copilot Suggestions Shown / Accepted	14,800 shown / 9,768 accepted (Accept Rate: 66.0%)
Cursor Tokens Used	N/A (not activated)
PRs Opened / Merged	18 opened / 15 merged (PR Merge Rate: 83.3%)
Org Leaderboard Rank	#3 — Active | 3 PRs rejected — AI code review flags; coaching recommended

User 4 | Maya Thornton — Frontend Engineer III, Frontend Product

Metric	Value
Team	Frontend Product
Primary AI Tool	GitHub Copilot
Total Commits (30d)	58 commits
Total LoC Written	17,200 LoC

AI-Generated LoC	8,084 LoC (47.0% of total)
Manual LoC	9,116 LoC (53.0% of total)
AI LoC Merged to Main	5,901 LoC (AI Merge Rate: 73.0%)
Copilot Tokens Used	420,000 tokens
Copilot Suggestions Shown / Accepted	10,200 shown / 5,916 accepted (Accept Rate: 58.0%)
Cursor Tokens Used	N/A (not activated)
PRs Opened / Merged	14 opened / 12 merged (PR Merge Rate: 85.7%)
Org Leaderboard Rank	#4 — Active | Below-avg accept rate; Cursor trial recommended for component workflows

User 5 | Daniel Kim — ML Engineer, Data & ML Engineering

Metric	Value
Team	Data & ML Engineering
Primary AI Tool	GitHub Copilot
Total Commits (30d)	44 commits
Total LoC Written	21,400 LoC
AI-Generated LoC	7,276 LoC (34.0% of total)
Manual LoC	14,124 LoC (66.0% of total)
AI LoC Merged to Main	4,365 LoC (AI Merge Rate: 60.0%)
Copilot Tokens Used	868,500 tokens
Copilot Suggestions Shown / Accepted	5,200 shown / 2,756 accepted (Accept Rate: 53.0%)
Cursor Tokens Used	N/A (not activated)
PRs Opened / Merged	11 opened / 8 merged (PR Merge Rate: 72.7%)
Org Leaderboard Rank	#5 — Active | ML-specific code reduces AI relevance; Cursor exploration in progress

User 6 | Priya Nair — Data Engineer, Data & ML Engineering [AI License Idle]

Metric	Value
Team	Data & ML Engineering
Primary AI Tool	None (Copilot license assigned, not activated — 30 days idle)

Total Commits (30d)	39 commits
Total LoC Written	17,400 LoC
AI-Generated LoC	0 LoC (0.0% of total)
Manual LoC	17,400 LoC (100.0% of total)
AI LoC Merged to Main	N/A
Copilot Tokens Used	0 tokens
Cursor Tokens Used	0 tokens
PRs Opened / Merged	10 opened / 8 merged (PR Merge Rate: 80.0%)
Org Leaderboard Rank	#6 — License Idle | Manager action: schedule onboarding. Est. uplift: +8,700 AI LoC/mo

13.4 User Comparison Summary — Org Leaderboard

Rank	User	Team	AI Code %	AI Merge Rate	Tokens Used	Primary Tool	Status
#1	Jordan Lee	Platform Eng.	70.0%	86.0%	1,120,000	Cursor	Power User
#2	Alex Reyes	Platform Eng.	54.0%	84.0%	1,030,000	Cursor + Copilot	Power User
#3	Sam Okafor	Frontend Product	56.0%	76.0%	580,000	Copilot	Active
#4	Maya Thornton	Frontend Product	47.0%	73.0%	420,000	Copilot	Active
#5	Daniel Kim	Data & ML	34.0%	60.0%	868,500	Copilot	Active
#6	Priya Nair	Data & ML	0.0%	N/A	0	None	License Idle

13.5 30-Day Weekly Trend (Org-Wide)

Week	AI-Generated LoC	Manual LoC	AI Code %	AI Merge Rate
Week 1 (Feb 22 - Mar 1)	12,800	17,200	42.7%	68.2%
Week 2 (Mar 2 - Mar 8)	14,100	16,400	46.2%	71.0%

Week 3 (Mar 9 - Mar 15)	16,200	15,800	50.6%	74.5%
Week 4 (Mar 16 - Mar 20)	18,052	14,248	55.9%	79.3%

Trend: AI code share has grown from 42.7% to 55.9% across the 4-week period, with merge rate improving in tandem — suggesting growing reviewer confidence in AI-generated output as team familiarity increases.

AI Code Intelligence Dashboard | Product Requirements Document | v1.0 | March 2026
Prepared for Claude Code Agentic Build | Confidential